Technical Interview Guide

agentic-nlq-eval stack - 2027 standards

50 high-yield questions with detailed answers

and production-ready code snippets

Python 3.11+ · Pydantic · Text-to-SQL · LangGraph

Spark & DuckDB · sqlglot · Evaluation science · Testing

How to use this guide

Questions are grouped by difficulty. Within each band, categories rotate so you practice conceptual, architectural, and coding prompts. Answers are written to the depth expected in senior interviews in 2026-2027: trade-offs, failure modes, and runnable patterns-not buzzwords.

Stack context: a research harness that compares S1 (single-pass text-to-SQL), S2 (error-driven retry), and S3 (LangGraph multi-node pipeline with SQL validation and answer verification ablations).

Beginner (Q1-Q15)

Q1. What problem does a text-to-SQL (NLQ) system solve?

Difficulty: Beginner | Category: NLQ concepts

Answer. It lets non-SQL users ask questions in natural language; the system produces SQL, executes it against a relational schema, and often narrates the result. The hard parts are schema grounding, correct joins/filters, and not inventing columns.

Q2. Why keep the API key out of config hashes and logs?

Difficulty: Beginner | Category: Security / config

Answer. Secrets must never be persisted in reproducible fingerprints or experiment diaries. In this project the key is read from the environment at call time via an env var name stored in config, so identical experiments share a config hash without embedding credentials.

```
# Conceptual pattern
api_key_env = "NLQ_LLM_API_KEY"  # stored in config
api_key = os.environ.get(api_key_env)  # read at call time only
# Never put api_key into AppConfig fields that are hashed or logged.
```

Q3. What is the difference between S1 and S2 in this harness?

Difficulty: Beginner | Category: Architecture

Answer. S1 generates SQL once, validates/executes, then narrates-no retry. S2 feeds validator and execution errors back into the next SQL generation, up to a shared retry cap (three retries after the first attempt). Same model and prompts; different control loop.

Q4. Explain Parquet in one interview-ready sentence, then one practical benefit.

Difficulty: Beginner | Category: Data engineering

Answer. Parquet is a columnar on-disk format for analytics tables. Benefit: engines like DuckDB/Spark can read only needed columns and prune row groups, which matters when TPC-H lineitem grows to tens of millions of rows.

Q5. What does temperature=0 usually buy you in evaluation harnesses?

Difficulty: Beginner | Category: LLM ops

Answer. Lower temperature reduces sampling randomness, improving run-to-run stability for benchmarking. It does not guarantee determinism across providers or versions, so the harness still uses repeats and majority voting.

Q6. Why use Pydantic models for configuration?

Difficulty: Beginner | Category: Python

Answer. Pydantic validates types and constraints at load time (e.g., positive scale factors, frozen configs), fails fast on typos via extra='forbid', and serializes cleanly for hashing.

```
from pydantic import BaseModel, ConfigDict, Field

class PricingSettings(BaseModel):
    model_config = ConfigDict(extra='forbid', frozen=True)
```

```
    input_usd_per_million: float = Field(ge=0)
    output_usd_per_million: float = Field(ge=0)
```

Q7. What is a "gold" answer in a benchmark?

Difficulty: Beginner | Category: Evaluation

Answer. A trusted reference result for a question at a reference scale. Systems are scored by comparing their execution output (normalized) to gold. This project also cross-checks gold SQL on Spark and DuckDB to catch engine drift.

Q8. DuckDB vs Spark: when would you pick each in a research harness?

Difficulty: Beginner | Category: Engines

Answer. DuckDB: fast local analytics, great for gold checks and small/medium Parquet. Spark: answers at larger scales and matches production-like distributed SQL; needs a JVM. This harness uses Spark for answering runs and DuckDB heavily in validation/tests.

Q9. What is JSONL and why use it for experiment logs?

Difficulty: Beginner | Category: Observability

Answer. JSON Lines stores one JSON object per line. You can append safely, stream partial files, and resume by scanning completed run IDs without loading a giant nested document.

Q10. Write a tiny function that estimates USD cost from token counts.

Difficulty: Beginner | Category: Coding

Answer. Cost is linear in tokens per million for input and output rates-matching the harness BudgetGuard.

```
def estimate_cost(input_tokens, output_tokens, in_per_m, out_per_m):
    return (input_tokens / 1_000_000) * in_per_m + \
           (output_tokens / 1_000_000) * out_per_m

assert abs(estimate_cost(1_000_000, 500_000, 1.0, 2.0) - 2.0) < 1e-9
```

Q11. What is schema grounding?

Difficulty: Beginner | Category: NLQ concepts

Answer. Mapping question phrases to actual tables/columns (and join paths) that exist in the allowed schema. Failures here cause invented columns or wrong grain. S3 has an explicit schema_grounding node before SQL generation.

Q12. Why expose business views instead of raw TPC-H names to the model?

Difficulty: Beginner | Category: Product / NLQ

Answer. Raw warehouse names are cryptic and leak benchmark identity. Business views (customers, order_lines, ...) mirror how analysts think and keep profit definitions centralized, reducing inconsistent formulas across systems.

Q13. What does pytest's pythonpath = ["src"] accomplish in pyproject.toml?

Difficulty: Beginner | Category: Testing

Answer. It puts the src layout on sys.path so tests can import nlq without installing the package in editable mode for every CI job-common for src-layout projects.

Q14. Explain SELECT-only validation in one paragraph.

Difficulty: Beginner | Category: Security

Answer. Before execution, parse SQL and reject DDL/DML/multiple statements so the model cannot drop tables or mutate data. This harness uses sqlglot with a Spark dialect and also checks tables/columns against the business schema and bans unconstrained cross joins.

Q15. Implement a safe path join that rejects path traversal for log filenames.

Difficulty: Beginner | Category: Coding

Answer. Resolve the candidate path and require it to remain under the runs directory-same idea as the demo server's _is_safe_log.

```
from pathlib import Path

def safe_log_path(runs_dir: Path, name: str) -> Path:
    path = (runs_dir / name).resolve()
    path.relative_to(runs_dir.resolve())  # raises if outside
    if not path.is_file() or path.suffix != ".jsonl":
        raise FileNotFoundError(name)
    return path
```

Intermediate (Q16-Q35)

Q16. Design a stable run_id for resumable experiments. What fields belong in the hash?

Difficulty: Intermediate | Category: Experiment design

Answer. Include experiment number, system, question, scale, repeat, warmup flag, config hash, model, and prompts version. Exclude wall-clock timestamps and secrets. Resume skips IDs already present as question_run lines in JSONL.

```
import hashlib

def run_id_for(**parts: str) -> str:
    material = "|".join(f"{k}={parts[k]}" for k in sorted(parts))
    return hashlib.sha256(material.encode()).hexdigest()[:32]
```

Q17. Why majority-vote repeats before McNemar's test?

Difficulty: Intermediate | Category: Statistics

Answer. Three stochastic repeats of the same question are not three independent samples of the population of questions. Collapsing to one vote per question avoids pseudo-replication. Ties can be treated as incorrect (as this project does).

Q18. Sketch a BudgetGuard that fails closed before the next LLM call.

Difficulty: Intermediate | Category: Coding

Answer. Charge after each call; before_call raises if spent already meets/exceeds the cap. Acknowledge that an in-flight call can still finish over budget because tokens are known only afterward.

```
class BudgetExceededError(Exception):
    pass

class BudgetGuard:
    def __init__(self, cap_usd):
        self.cap_usd = cap_usd
        self.spent_usd = 0.0

    def before_call(self):
        if self.cap_usd is not None and self.spent_usd >= self.cap_usd:
            raise BudgetExceededError('budget reached')

    def charge(self, cost: float):
        self.spent_usd += cost
```

Q19. How does LangGraph differ from a linear Python script of LLM calls?

Difficulty: Intermediate | Category: Agents / LangGraph

Answer. LangGraph models a stateful graph with nodes and conditional edges (retry loops, ablation shortcuts). State accumulates fields (SQL, feedback, timings). You get explicit control flow, recursion limits, and easier ablations by changing edges rather than rewriting node code-as S3-V/S3-A do.

Q20. Show a minimal StateGraph pattern for validate-or-retry SQL generation.

Difficulty: Intermediate | Category: Coding

Answer. Conditional edges after validation either execute or return to generation until retries exhaust.

```
from langgraph.graph import StateGraph, START, END

def build(enable_validation: bool):
    g = StateGraph(dict)
    g.add_node('sql_generation', generate)
```

```
    g.add_node('sql_validation', validate)
    g.add_node('execute', execute)
    g.add_edge(START, 'sql_generation')
    g.add_conditional_edges(
        'sql_generation',
        lambda s: 'execute' if not enable_validation else 'sql_validation',
        {'execute': 'execute', 'sql_validation': 'sql_validation'},
    )
    g.add_edge('execute', END)
    return g.compile()
```

Q21. What should a deterministic SQL validator check beyond "parses"?

Difficulty: Intermediate | Category: Architecture

Answer. Single statement; read-only; tables/columns subset of schema; resolve aliases/CTEs; reject ambiguous columns; reject CROSS JOIN / join without ON/USING. Keep it model-free so messages are safe feedback for retries.

Q22. Implement a sqlglot-based SELECT-only gate (Spark dialect).

Difficulty: Intermediate | Category: Coding

Answer. Parse with read='spark', require one Select/Union-family statement.

```
from sqlglot import exp, parse

READ_ONLY = (exp.Select, exp.Union, exp.Intersect, exp.Except)

def is_single_select(sql: str):
    try:
        stmts = [s for s in parse(sql, read='spark') if s is not None]
    except Exception as e:
        return False, f'parse error: {e}'
    if len(stmts) != 1:
        return False, 'Exactly one SQL statement is allowed'
    if not isinstance(stmts[0], READ_ONLY):
        return False, 'Only a SELECT query is allowed'
    return True, 'ok'
```

Q23. How do you score accuracy when evaluating at a scale different from gold YAML?

Difficulty: Intermediate | Category: Evaluation

Answer. Re-run the gold SQL at the evaluation scale and compare the system's result to that dynamic gold. Keep YAML values as the SF0.01 audit trail. Normalize types, rounding, and column order per question metadata.

Q24. Define claim-level faithfulness labels used in this project.

Difficulty: Intermediate | Category: Evaluation

Answer. supported, contradicted, unsupported, non_verifiable. Numbers/comparisons are checked against logged results; explanatory claims without checkable numbers stay non_verifiable; inventing a new verification query is avoided-unsupported records that no extra query was run.

Q25. Write a claim number extractor and membership check against result cells.

Difficulty: Intermediate | Category: Coding

Answer. A minimal faithfulness building block: extract numbers from claim text and see if they appear in normalized cells.

```
import re

_NUM = re.compile(r'-?\d+(?:\.\d+)?')

def numbers_in(text: str):
    return _NUM.findall(text)
```

```
def claim_supported_by_cells(claim, cells):
    cell_set = set(cells)
    found = numbers_in(claim)
    return bool(found) and all(n in cell_set for n in found)
```

Q26. Why clear Spark cache between measured runs and log cache state?

Difficulty: Intermediate | Category: Performance / fairness

Answer. Cached tables can make later systems look faster. Clearing between runs reduces cross-run contamination; logging cache state documents residual effects for analysis.

Q27. How would you structure PromptLibrary so JSON braces in templates are safe?

Difficulty: Intermediate | Category: LLM ops

Answer. Avoid str.format on whole prompts. Use explicit placeholders like <<SCHEMA>> and replace them. Version prompts via prompts/VERSION written into every run record.

```
def render(template: str, **tokens: str) -> str:
    out = template
    for key, value in tokens.items():
        out = out.replace(f'<<{key.upper()}>>', value)
    return out
```

Q28. Describe a production-ready OpenAI-compatible client without the official SDK.

Difficulty: Intermediate | Category: Coding

Answer. Use urllib against /v1/chat/completions, map HTTP 408/429/5xx to retryable errors with tenacity, return text + token counts + latency. Keep temperature and model from config.

```
import json, time, urllib.request

def chat_complete(url, api_key, model, messages, temperature=0.0):
    body = json.dumps({
        'model': model, 'temperature': temperature,
        'messages': messages,
    }).encode()
    req = urllib.request.Request(
        url, data=body, method='POST',
        headers={'Authorization': f'Bearer {api_key}',
                 'Content-Type': 'application/json'})
    t0 = time.perf_counter()
    with urllib.request.urlopen(req, timeout=60) as resp:
        payload = json.loads(resp.read().decode())
    latency = time.perf_counter() - t0
    text = payload['choices'][0]['message']['content']
    usage = payload.get('usage') or {}
    return text, usage.get('prompt_tokens', 0), \
           usage.get('completion_tokens', 0), latency
```

Q29. What is an ablation study in agent systems, using S3-V and S3-A?

Difficulty: Intermediate | Category: Architecture

Answer. Disable one component at a time while keeping the rest fixed. S3-V disables SQL validation (first SQL executes); S3-A disables answer verification. Comparing to full S3 estimates each check's contribution to accuracy/faithfulness/latency.

Q30. Implement seeded shuffle planning so systems are interleaved.

Difficulty: Intermediate | Category: Coding

Answer. Build the Cartesian product of question x system x scale x repeat, shuffle with Random(seed), insert warmup.

```
import random

def plan_runs(questions, systems, scales, repeats, seed):
    planned = [
        {'q': q, 'system': s, 'scale': sc, 'repeat': r, 'warmup': False}
        for q in questions for s in systems for sc in scales
        for r in range(1, repeats + 1)
    ]
    rng = random.Random(seed)
    rng.shuffle(planned)
    planned.insert(0, {**planned[0], 'repeat': 0, 'warmup': True})
    return planned
```

Q31. How do you keep unit tests from calling a real LLM API?

Difficulty: Intermediate | Category: Testing

Answer. Inject a scripted LLMClient that returns predetermined JSON for each node/prompt. Tests still exercise DuckDB/Spark locally. Assert on SQL validation, scoring, and graph routing without network or spend.

Q32. Normalize numeric cells for comparison: show half-even rounding.

Difficulty: Intermediate | Category: Coding

Answer. Use decimal with ROUND_HALF_EVEN when question metadata demands banker's rounding.

```
from decimal import Decimal, ROUND_HALF_EVEN

def round_half_even(value: float, places: int) -> Decimal:
    q = Decimal('1').scaleb(-places)
    return Decimal(str(value)).quantize(q, rounding=ROUND_HALF_EVEN)

assert round_half_even(2.5, 0) == Decimal('2')
assert round_half_even(3.5, 0) == Decimal('4')
```

Q33. Windows + Spark + spaces in OneDrive paths: what goes wrong and how do you mitigate?

Difficulty: Intermediate | Category: Platform

Answer. JVM/Spark often break on paths with spaces. Mitigation used here: rewrite SPARK_HOME and data paths to 8.3 short paths before starting the JVM so CI and laptops with OneDrive layouts still run.

Q34. Config load precedence: YAML, dotenv, process env-who wins?

Difficulty: Intermediate | Category: Config

Answer. Load YAML defaults, apply .env without clobbering variables already set in the process, then apply explicit NLQ_* overrides. Document precedence; test it. Empty NLQ_BUDGET_USD clears the cap.

Q35. Write a stratified sampler: up to N questions per tier.

Difficulty: Intermediate | Category: Coding

Answer. Group by tier, shuffle each pool with a seed, take min(N, available)-matches experiment 2's scaffold behavior.

```
import random
from collections import defaultdict

def stratified(questions, per_tier, seed):
    buckets = defaultdict(list)
    for q in questions:
        buckets[q['tier']].append(q)
    rng = random.Random(seed)
```

```
    out = []
    for tier in sorted(buckets):
        pool = buckets[tier][:]
        rng.shuffle(pool)
        out.extend(pool[:per_tier])
    return out
```

Advanced (Q36-Q50)

Q36. Architect a fair comparison between agentic and single-pass text-to-SQL. Name five controls.

Difficulty: Advanced | Category: Architecture

Answer. Controls: (1) same model+temperature, (2) same schema prompt surface, (3) same narration prompt, (4) shared SQL validator rules where applicable, (5) seeded interleaved schedule + warmup discard + cache clear + identical row caps/timeouts. Log config hash and prompt version on every run.

Q37. Where can agentic pipelines still hallucinate despite SQL validation?

Difficulty: Advanced | Category: Failure modes

Answer. Valid SQL can be semantically wrong (wrong filter year, wrong grain, subtle join). Narration can invent claims not in the result set. Answer verification and claim-level faithfulness catch some of this; execution accuracy against gold catches others. Validation is not correctness.

Q38. Implement Holm-Bonferroni adjustment for a list of p-values.

Difficulty: Advanced | Category: Coding / stats

Answer. Sort p-values ascending; adjust with (m-i+1) factor; enforce monotonicity.

```
def holm(pvals):
    m = len(pvals)
    order = sorted(range(m), key=lambda i: pvals[i])
    adj = [0.0] * m
    prev = 0.0
    for rank, i in enumerate(order):
        val = min(1.0, (m - rank) * pvals[i])
        val = max(val, prev)
        adj[i] = val
        prev = val
    return adj
```

Q39. Design result truncation safely (row_cap=200) without silent wrong answers.

Difficulty: Advanced | Category: Systems design

Answer. Cap protects memory/latency, but aggregates must be computed in SQL-not client-side on truncated rows. Log truncation flags. Prefer pushing GROUP BY/ORDER BY/LIMIT into the engine. For entity lists, document that narratives based on capped rows may be incomplete.

Q40. Show a typed SystemResult assembly pattern after SQL + narration.

Difficulty: Advanced | Category: Coding

Answer. Centralize assembly so S1/S2/S3 share logging fields: SQL, validation, rows, narrative, retries, timings, failure.

```
from dataclasses import dataclass, field

@dataclass
class SystemResult:
    sql: str
    narrative: str
    answer_values: list
    retries: int = 0
    failed: bool = False
    exec_time_s: float = 0.0
    llm_calls: list = field(default_factory=list)

def assemble(sql, narrative, values, **meta):
    return SystemResult(sql=sql, narrative=narrative,
                        answer_values=values, **meta)
```

Q41. How would you detect flaky gold disagreements between Spark and DuckDB?

Difficulty: Advanced | Category: Evaluation eng

Answer. Normalize schemas/types, compare sorted multisets or ordered frames per question flags, hash canonical CSV of results, fail CI on mismatch. Investigate dialect functions, null sorting, timestamp precision, and floating tolerance metadata.

Q42. Propose a circuit-breaker around LLM retries distinct from SQL retries.

Difficulty: Advanced | Category: Reliability

Answer. SQL retries are semantic (bad query). API retries are transport (429/5xx) via tenacity with jittered backoff and max_api_retries. Do not conflate them in one counter-or a transient 429 will burn your SQL attempt budget.

```
from tenacity import retry, stop_after_attempt, \
    wait_random_exponential, retry_if_exception_type

class TransientLLMError(Exception):
    pass

@retry(reraise=True, stop=stop_after_attempt(5),
       wait=wait_random_exponential(multiplier=1, max=20),
       retry=retry_if_exception_type(TransientLLMError))
def complete_with_transport_retries(client, messages):
    return client.complete(messages)
```

Q43. Cluster bootstrap for faithfulness intervals-why resample whole questions?

Difficulty: Advanced | Category: Statistics

Answer. Claims nested inside questions are dependent. Resampling claims i.i.d. understates variance. Cluster bootstrap resamples questions (clusters) to keep within-question dependence, matching how the harness computes faithfulness intervals.

Q44. Implement a simple cluster bootstrap CI for a mean rate per question.

Difficulty: Advanced | Category: Coding

Answer. Each question contributes one rate; resample questions with replacement.

```
import random, statistics

def cluster_bootstrap_ci(rates, n_boot=2000, alpha=0.05, seed=0):
    rng = random.Random(seed)
    n = len(rates)
    stats = []
    for _ in range(n_boot):
        sample = [rates[rng.randrange(n)] for _ in range(n)]
        stats.append(statistics.fmean(sample))
    stats.sort()
    lo = stats[int(alpha/2 * n_boot)]
    hi = stats[int((1 - alpha/2) * n_boot)]
    return statistics.fmean(rates), lo, hi
```

Q45. How do recursion_limit and max_sql_retries interact in LangGraph S3?

Difficulty: Advanced | Category: Agents

Answer. max_sql_retries bounds the business loop; recursion_limit is a framework safety net against unbounded graphs. Set recursion_limit high enough for worst-case node visits (understanding to retries to revisions) but still finite. Prefer explicit counters in state over relying only on recursion_limit.

Q46. Design provenance for answers so faithfulness checks are auditable.

Difficulty: Advanced | Category: Architecture

Answer. Store SQL, truncated result snapshot, evidence items (row refs/aggregates), raw model texts, and claim verdicts in the question_run record. Narration must cite evidence IDs. Verification compares claims to logged evidence-not to a fresh silent query-unless explicitly designed otherwise.

Q47. Write an idempotent JSONL resume checker.

Difficulty: Advanced | Category: Coding

Answer. Scan existing file for question_run IDs; skip planned runs already completed.

```
import json
from pathlib import Path

def completed_ids(path: Path) -> set[str]:
    done = set()
    if not path.is_file():
        return done
    with path.open(encoding='utf-8') as f:
        for line in f:
            rec = json.loads(line)
            if rec.get('record_type') == 'question_run':
                done.add(rec['run_id'])
    return done

def should_run(run_id, done, resume):
    return not (resume and run_id in done)
```

Q48. Discuss privacy and compliance when logging raw model text in NLQ evals.

Difficulty: Advanced | Category: Governance

Answer. Raw outputs aid reproducibility but may contain sensitive question content or regurgitated rows. Mitigations: local-only runs dirs, gitignore data/runs/results, no secrets in logs, retention policy, redact PII fields for taxi data, separate demo fixtures from production extracts.

Q49. Optimize experiment 2 cost without destroying statistical validity.

Difficulty: Advanced | Category: Experiment design

Answer. Stratify tiers; cap questions per tier; use cheaper scales for smoke; reserve SF10 for a subset; temperature 0; resume; budget guard; majority-vote 3 repeats rather than 10; pre-validate SQL to avoid paying for doomed executions-but report that policy in the paper.

Q50. System design prompt: outline modules for an NLQ eval harness like this one.

Difficulty: Advanced | Category: Architecture

Answer. Modules: config (validated, hashed), data (TPC-H/NYC + business views + schema context), engines (Spark/DuckDB adapters + QueryResult), benchmark (YAML questions + gold validate), llm (client, prompts, cost guard), systems (S1/S2/S3 graph), eval (accuracy, faithfulness, latency), logging (JSONL), analysis (stats/plots/reproduce), tests (scripted LLM). Keep secrets out of hashes; make ablations flags, not forks.

```
nlq/
  config.py
  logging_utils.py
  data/       # generate_tpch, views, schema_context, nyc_tlc
  engine/     # spark_engine, duckdb_engine, results
  benchmark/  # schema, questions/*.yaml, validate_gold
  llm/        # base, provider, prompts, cost
  systems/    # s1_single, s2_retry, s3_agentic/, sql_validation
  eval/       # accuracy, faithfulness, latency, run_experiment
  analysis/   # stats, plots, reproduce
  demo/       # optional local server
```

Appendix - Quick study map

- Beginner: NLQ basics, config/secrets, S1 vs S2, Parquet, gold answers, JSONL.
- Intermediate: LangGraph edges, sqlglot validation, budget guards, stratified plans, faithfulness labels.
- Advanced: fair experiment controls, Holm/bootstrap, provenance, resume semantics, governance.

Practice tip: re-implement S1 and a tiny validate-retry loop in under 100 lines, then explain what S3's extra nodes buy you on semantic errors that still pass SQL validation.